

PULSE Instrument Cluster

Ahmed Abdelghany

2026-09-23

Contents

Document control	1
Change log	1
1. Purpose	1
2. The Flutter cluster	2
2.1 Data flow	2
2.2 Design rules	2
2.3 Where the Compose cluster differs	2
3. The providers	2
3.1 Shared module: broker.py	3
3.2 Publishing rules	3
3.3 The driver monitor	3
4. What this document does not cover	3

Document control

Field	Value
Document ID	PULSE-SWAD-001
Title	Software Architecture Description
Version	0.1
Status	Draft for the sprint 2 review
Date	2026-09-23
Author	Ahmed Abdelghany
Reviewer	Cockpit Electronics / HMI Platform Group (sponsor)
Review gate	Sprint 2 review, week 6, 25 September 2026
Parent	PULSE-SAD-001
Related	PULSE-SyRS-001, PULSE-SAF-001, PULSE-SEC-001

Change log

Version	Date	Author	Change
0.1	2026-09-23	A. Abdelghany	First draft: one page each for the Flutter cluster and the providers, as PULSE-SAD-001 section 1 promised for sprint 2 (SR-10)

1. Purpose

PULSE-SAD-001 describes the system: what the components are and how they meet at the broker. This document goes one level down, inside two of those components, for a reader who has to change them. One page each: the Flutter cluster, and the Python providers. The Compose cluster mirrors the Flutter one and is covered by the same page where it differs.

Abbreviations are defined in the glossary, PULSE-StRS-001 section 8.

2. The Flutter cluster

Source: pulse-cluster/lib/. Three hand-written layers and one generated one.

Layer	Files	Owns
Transport	vss_service.dart	One gRPC subscription to the broker, reconnect, credentials, catalogue limits
State	vehicle_state.dart	Every displayed value, arrival times, freshness, degraded forms
Presentation	screen/cluster_pager.dart, screen/pulse_screen.dart, screen/classic_screen.dart	Drawing only. No signal names, no thresholds
Generated	generated/kuksa/val/v1/	Protocol stubs from the KUKSA .proto files. Never edited

2.1 Data flow

One value takes this path, and only this path:

1. VssService holds one streaming Subscribe call for all 28 paths in PULSE-SAD-001 appendix A. Before subscribing it reads each path's catalogue restriction from the broker, so the limits it enforces are the catalogue's, not its own (IR-2).
2. Each update is checked against that restriction. A value outside it is refused and counted, and does not refresh the signal's arrival time (CS-5). An accepted value is written into VehicleState and stamped with the monotonic arrival time.
3. VehicleState is a ChangeNotifier. Every write notifies; a 100 ms ticker also notifies whenever any safety signal crosses its freshness criterion, so a lost signal degrades even when nothing arrives (SM-1).
4. The screens read *display-ready* values: speedText is already "—" when speed is stale, gearText is already "?", alertShown is already NO_FACE or UNAVAILABLE. A widget never decides whether a value is trustworthy; the state has decided before the widget sees it.

2.2 Design rules

- **The catalogue is the only source of semantics.** Paths appear once, in VssService; units and ranges come from the broker at run time.
- **Freshness is judged by arrival, not by the broker's timestamp.** That needs no clock agreement between hosts, and it catches a lost broker or link as well as a stalled provider. Providers therefore republish safety signals every tick even when unchanged.
- **Degraded forms are fixed in VehicleState**, one place, from PULSE-SAF-001 section 3. A stale monitor is never shown as "no alert".
- **Connection loss marks everything stale at once**, in the same finally that closes the channel.
- **Credentials are found, never embedded.** The service locates .run/pki by walking up from the working directory and from the executable, so run.sh and a bare bundle both find it, and refuses to connect without it unless PULSE_INSECURE=1 says the broker was started that way.

2.3 Where the Compose cluster differs

pulse-cluster-android/app/src/main/java/com/pulse/cluster/. Same three layers: VssClient.kt (transport and limits), VehicleUi (an immutable data class replacing the notifier; each update produces a new copy), and PulseScreen.kt. Freshness is evaluated against a nowNs the activity refreshes every 100 ms. Credentials come from the app's external files directory, pushed by scripts/android-push-pki.sh. It subscribes to 17 of the 28 paths: it has no driver-monitoring panel and no intervention overlays, which is the divergence recorded against FR-13.

3. The providers

Source: emulator/. Python, one process per provider, sharing one module.

Provider	File	Publishes	Rate
Vehicle simulator	telemetry_sim.py	Drive cycle: speed, engine speed, gear, fuel, tyres, lap and motorsport data, the minimum-risk manoeuvre state	10 Hz fast group, 1 Hz slow group
Driver monitor	dms.py	Fatigue, distraction, attention, eyes-on-road, active flag, alert state	10 Hz publish, 15 Hz processing
Engine audio	engine_audio.py	Nothing. A consumer that synthesises engine sound from engine speed	subscribes
CAN feeder (spike)	scripts/can_spike.py	Whatever the AGL DBC maps: speed, engine speed, hazard state in the spike	as frames arrive

3.1 Shared module: broker.py

Every provider reaches the broker through `connect(role)`. It returns a client configured for TLS against the development CA with the token for that role, or a plaintext client when `PULSE_INSECURE=1`. A provider that cannot find its token stops with a message naming the fix rather than falling back silently. Roles and their scopes are minted by `scripts/dev_pki.py`: each provider may publish only the signals its source declares, and consumers may only read (CS-4).

3.2 Publishing rules

- **Changed values only, except safety signals.** `send_if_changed` skips a value equal to the last one sent, to keep the bus quiet. The signals in PULSE-SAF-001 section 3 are excepted through a `KEEPALIVE` set and go every tick, because the HMIs judge freshness by arrival and a steady speed must still arrive.
- **Catalogue units, always.** The simulator integrates the odometer in kilometres for convenience and converts to metres at the publish call, because that is the catalogue's unit. The unit-sanity regression check exists because this was once wrong and nothing else could see it.
- **The safety loop is in the simulator, not the monitor.** `dms.py` only reports the driver's state. `telemetry_sim.py` subscribes to that state and runs the minimum-risk manoeuvre (PULSE-SAF-001 SM-3) as a vehicle would. The monitor cannot brake the car; it can only be believed or not.

3.3 The driver monitor

`dms.py` is a pipeline: camera frame, face landmarks (MediaPipe on the CPU, no accelerator), behaviour metrics (eye closure over time, yawns, head pose), a small state machine with hold and clear times (FR-19), one publish. No frame is written or sent anywhere; the privacy trace in the regression suite asserts that (CS-6). It needs AVX instructions, which some virtual machines do not expose; on such a host it cannot start, and the cluster shows "monitoring unavailable" rather than a plausible blank.

4. What this document does not cover

Build and packaging, which are in `README.md` and `ONBOARDING.md`, and the broker itself, which is upstream KUKSA and described in PULSE-SAD-001 section 5.